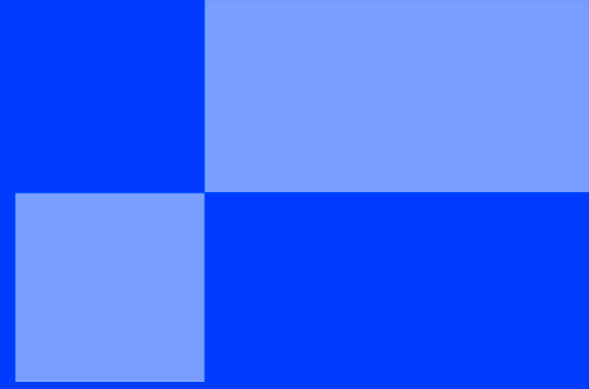
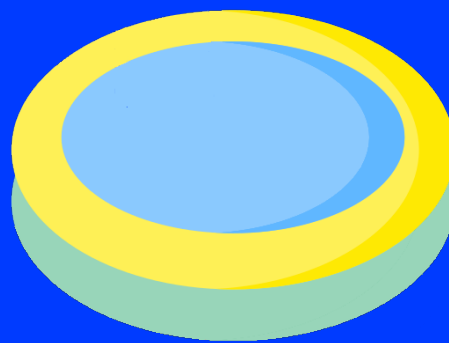




PREPARATION BOOTCAMP

< LISTS & DICTIONARIES />



PREPARATION BOOTCAMP

GAME



If you need a little break in your day, we suggest you play [the bandit wargame](#). We also encourage you to resume your game later.



Lists

Task 1.1



Create a `list` of 5 elements. Then, print its first element.

Task 1.2



Display the last element of your list.



Your code must be functional whatever number of elements the list contains.

Task 1.3



Add the integer `42` at the end of your list. Then, add the string `forty-two` at the end of your list.

Task 1.4



Display your entire list. Then, display each element of your list one by one.

Task 1.5



Delete the last element of your list. Then, display your list to check if you did it properly.



Your code must be functional whatever number of elements the list contains.

Task 1.6



Add an element at the beginning of the list and display all its elements.

Task 1.7



Display the sub-list from the second to the fourth element (included).



Can you do it in one line?

Task 1.8



Create and display a new *reversed* list of your previous list, starting from the end.

Task 1.9



Add the ten integers from 11 to 20 at the end of your list.

Task 1.10

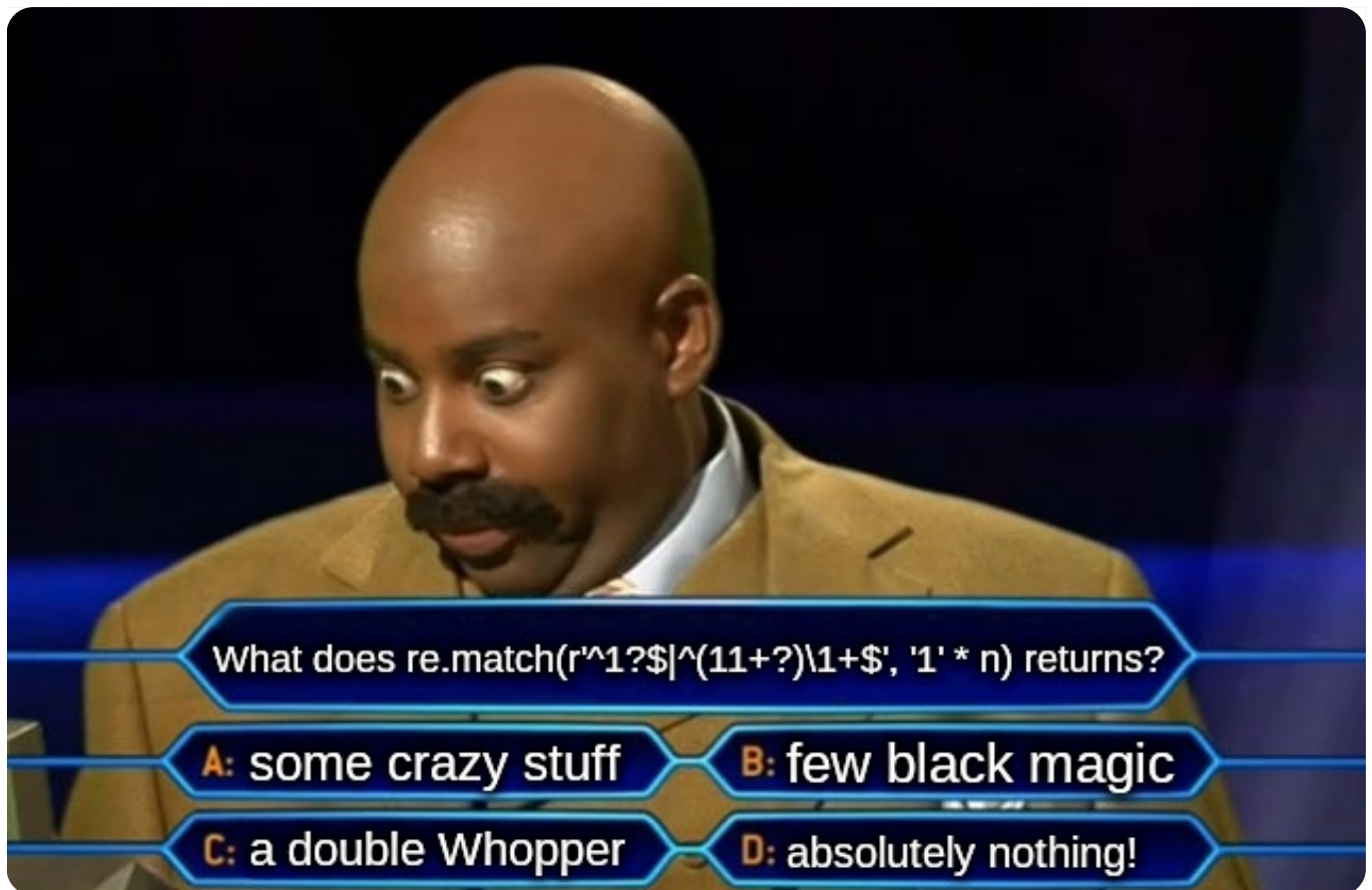


What does the following code do?

```
my_first_list = [4, 5, 6]
my_second_list = [1, 2, 3]
my_first_list.extend(my_second_list)
```

Same with:

```
my_first_list = [7, 8, 9]
my_second_list = [4, 5, 6]
my_first_list = [*my_first_list, *my_second_list]
```



Task 1.11



Create a list of 5 numbers. Then, print the result of the multiplication of all its elements.



The input [1, 2, 3, 4, 5] should return 120.

Task 1.12



Test this code and try to explain it: `[x + 10 for x in [3, 2, 6, 7, 1, 4]]`

Task 1.13



Create a list of 5 numbers. Then, display the smallest element. Finally, display the biggest element.

Task 1.14



Sort your list in descending order.

Task 1.15



Test this code and try to explain it: `[x // 2 if x % 2 == 0 else x * 2 for x in [42, 3, 4, 18, 3, 10]]`

Task 1.16

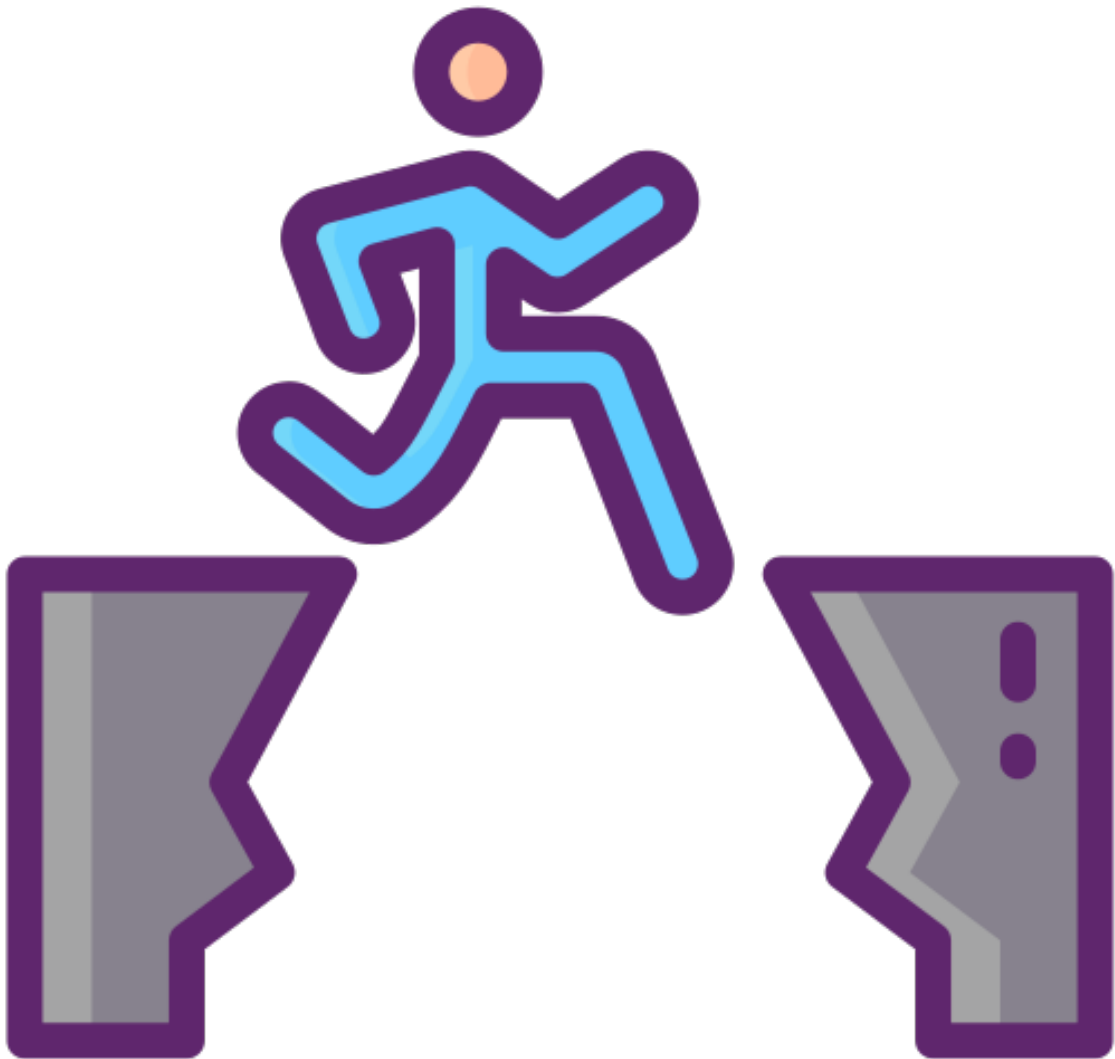


Write a program that deletes all the duplicated elements in a list.

Test it with [1,1,1,1,2,2,2,2,2] and with [42, '42', 42.0, 21+21, 42*10/10].
Both should return 2 elements.

CHALLENGE

Create a list of 1000000 random integers. Then, sort this list as fast as possible.



If you want to know how long the execution of your program lasted, you can:

- ✓ insert `start=time.time()` at the top of your code and `print(time.time()-start)` at the end;
- ✓ use the command `time YOUR_SCRIPT.py`.

Dictionaries

Task 2.1

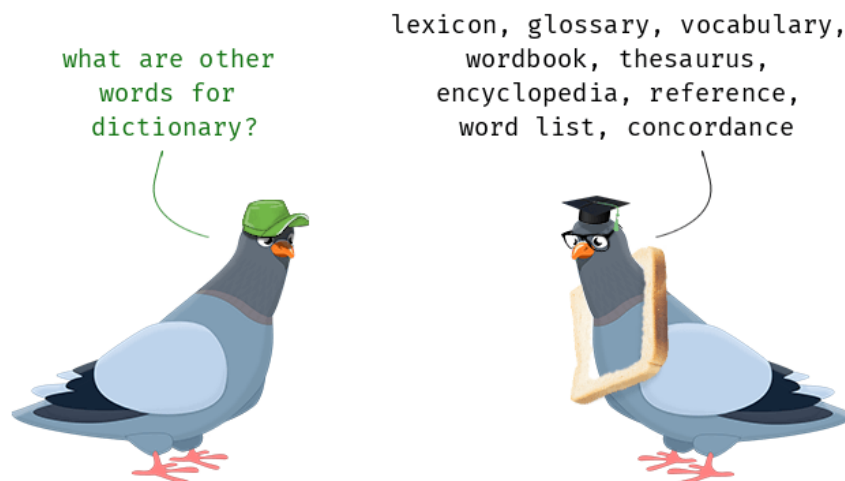


Create a **dictionary** stored in a `pokemons` variable. A dictionary must contain some key/value pair(s).

The keys are strings representing the names of some Pokemon: `Pikachu`, `Bulbasaur` and `Charmander`.

The values are strings representing their respective types: `Electric`, `Grass` and `Fire`.

Finally, print this dictionary.



Task 2.2



Inside `pokemons`, add the key `Blaziken` with the value `Fire`. Then, print this dictionary.

Task 2.3



In your dictionary `pokemons`, add the key `Pikachu` with the value `["Pichu", "Raichu"]`. Look at what is happening. What do you observe? What do you make of it?

Task 2.4



Let's do it again, but differently. Start by creating the dictionary `types`. The keys are strings representing Pokemon types (e.g., `'Electric'`, `'Grass'`, `'Fire'`). For each key, add an **empty** list as value (for now). Finally, print the dictionary.

Task 2.5



In the values/lists of your dictionary `types`, add each Pokemon from this list `["Pikachu", "Bulbasaur", "Charmander", "Leafeaon", "Scovillain"]` to the list of its type.

Task 2.6



Print all the keys (Pokemon types) contained in your previous dictionary.

Task 2.7



Using your previous dictionary, retrieve the type of "Pikachu".



Try to do this in two different ways.

Task 2.8



Store this dictionary into the variable `superheroes`.
Then, print the value of Superman's city.

```
{
  "Batman" : {
    "id": 1,
    "aliases": ["Bruce Wayne", "Dark knight"],
    "location": {
      "number" : 1007,
      "street": "Mountain Drive",
      "city": "Gotham"
    }
  },
  "Superman" : {
    "id": 2,
    "aliases": ["Kal-El", "Clark Kent", "The Man of Steel"],
    "location": {
      "number" : 344,
      "street": "Clinton Street",
      "apartment": "3D",
      "city": "Metropolis"
    }
  },
}
```



Task 2.9



Inside the dictionary `superheroes`, add:

- ✓ `Caped Crusader` inside `Batman's aliases`;
- ✓ a new superhero `Wolverine`, with 3 as `id`, and no aliases nor location.

Task 2.10



For each superhero contained in `superheroes`, enumerate all the aliases she/he has.
Your result should look like this:

```
Batman:
Bruce Wayne
Dark knight
Caped Crusader

Superman:
Kal-El
Clark Kent
The Man of Steel

Wolverine:
No aliases found.
```

Task 2.11



Inside this dictionary, get the key with the maximum value:

```
{
  "dalmatians": 101,
  "pi": 3.14,
  "beast": 666,
  "life": 42,
  "googol": 10^100,
  "jordan": 23,
  "life, the universe and everything": 42,
  "emergency": 911,
  "euler": 2.71828
}
```

CHALLENGE

Let's play Scrabble.

Without any bonus, the score for a word is calculated based on the letter values as follows:

- ✓ A, E, I, O, U, L, N, S, T, R = 1 point each;
- ✓ D, G = 2 points each;
- ✓ B, C, M, P = 3 points each;
- ✓ F, H, V, W, Y = 4 points each;
- ✓ K = 5 points;
- ✓ J, X = 8 points each;
- ✓ Q, Z = 10 points each.

Write a Python function that takes a string as input and returns the base score for that word by summing the scores of its letters.



If you do not feel comfortable now, have a look at the next exercises, then come back here. Using Python built-in functions will make your solution as clean and efficient as possible. And it should take very few lines of code.

v 2.2

{EPITECH}